

# Program of Thoughts Prompting: A Replication and Extension Study Using Open-Source Language Models

Himnisha Sai Kommana  
*School of AI*

*Amrita Viswa Vidyapeetham*  
Bengaluru, India

T Sai Swaroop  
*School of AI*

*Amrita Viswa Vidyapeetham*  
Bengaluru, India

V Dasa Manoj  
*School of AI*

*Amrita Viswa Vidyapeetham*  
Bengaluru, India

bl.en.u4aid23018@bl.students.amrita.edu bl.en.u4aid23051@bl.students.amrita.edu bl.en.u4aid23055@bl.students.amrita.edu

**Abstract**—LLMs have been shown to reason remarkably well Capabilities through chains-of-thought (CoT) prompts. Nonetheless, CoT expects the language model to reflect over both reasoning and arithmetics, which frequently results in numerical errors. Thought: Program (PoT) prompting solves this by generating a computer program that can be executed in Python and delegating the computation to an external interpreter. This project reproduces and extends the original PoT work by Chen et al. Llama-3 (2023) — Utilized free open-source models. 1-8B and Llama-3. 3-70B) on four mathematical benchmarks, namely GSM8K, SVAMP, AQuA-RAT and MultiArith via the Groq inference API. Our replication uncovers an important finding that was missing in the original paper: the advantage of PoT over CoT is highly dependent on what kind of backbone model we take. In the second setting, we discover for data up to 2023-October relatively complex multi-step problem solving datasets (e.g. GSM8K, 86 % vs 82 % zero-shot), that CoT outperforms PoT on the task of instruction-tuned chat models but PoT obtains perfect accuracy performance on MultiArith with a score of 100% and competitive scoring on SVAMP reaching scores with an average zero-shot number reaching up to 85.3%. Then we build upon the paper with five extensions: SymPy based symbolic answer verifier, complexity-aware methods router, self-consistency decoding, PoT-CoT hybrid pipeline and automated errors classifier. Results demonstrate that Computation offloading object detections can gain from in-domain training when the model generates valid programs with a high level of confidence, and program generation quality is identified as a largely limiting factor for chat-based models.

**Index Terms**—Large Language Models, Program of Thoughts, Chain-of-Thought, Prompt Engineering, Numerical Reasoning, Python Execution, SymPy, Groq API, Open-Source Models, Self-Consistency

## I. INTRODUCTION

With the recent advances in LLMs like GPT-3, Codex and Llama, the natural language understanding and mathematical reasoning have been revolutionized. Chain-of-Thought (CoT) prompting, which is a prompting that enables reasoning, it elicits intermediate steps from the model in order to arrive at an answer. CoT helps logical coherence, but also requires the model to keep performing arithmetic internally — predicting number tokens step-by-step, which makes errors on large numbers, polynomial equations and iterative computations.

Program of Thoughts (PoT) prompting Chen et al. This is addressed by (2023) by factoring for reasoning and computation separation. The model outputs a runnable program in python that articulates the reasoning process; all arithmetic is directed via a Python interpreter. The original paper showed around 12% average improvement on eight datasets over CoT using OpenAI Codex (code-davinci-002) as backbone.

Now Codex was dead in 2024. Before this work, it was an open question whether the benefits of PoT transfer to contemporary freely-available instruction-tuned chat models — which underlie most modern research in practice.

This work fills that gap in three contributions. Specifically, Llama models served by the free Groq API are used to replicate PoT experiments on GSM8K, SVAMP, AQuA-RAT and MultiArith. Second, we discover and characterise a model-type bias in PoT’s effectiveness that was not visible to the original paper: chat models prefer CoT, code-focused models prefer PoT. Third, we present five revisions that further or better frame PoT: symbolic proof with SymPy, complexity-aware routing, self-consistency decoding, a PoT+CoT hybrid pipeline, and automated error categorisation. The rest of this paper is organised as follows. II presents the problem statement. III reviews related work. IV describes the system architecture. V covers implementation. VI describes experiments. VII interprets results. VIII discusses advantages, limitations, and extensions. IX concludes.

## II. PROBLEM STATEMENT

Numerical reasoning, defined as ability to produce correct answers to questions involving arithmetic operations, algebra, iteration and symbolic calculation, is a necessary capability of general-purpose AI systems. Many applications that rely on correct and faithful numerical calculations include finance, science, automated mathematics and tutoring systems. LLMs exhibit fundamental limitations with numerical tasks. Because LLMs are built to produce natural language in token-by-token sequential steps, arithmetic is performed as part of language generation process and results in three key issues identified by Chen et al. (2023) and replicated by our experiments:

**Problem 1: Arithmetic Error Accumulation.** Arithmetic in CoT must be translated into natural language and its intermediate results generated via language modeling. If a calculation requires 50-step arithmetic on the chain of addends  $1, 2, 3, \dots$ , then even a 90% correct answer rate per step would be predicted for only  $(.9)^{50} < 1\%$  of generated answers in the end; arithmetic on a computer system is in contrast, exact.

**Problem 2: Inability to Solve Symbolic Equations.** CoT cannot execute standard algebraic procedures and cannot solve polynomial/differential equations, e.g. find interest rate  $x$  by computing from  $C = P(1+x)^N$  given  $P, C, N$ . For instance, asked to solve for the interest rate in compound interest formula, LLMs have no other choice but try to solve this problem with rough linguistic approximations leading to a failure in nearly all cases. Symbolic computation libraries like SymPy from Python, do execute them perfectly.

**Problem 3: Inefficient Iteration.** Iterating 50 steps requires the model to generate many more than a handful of natural language sentences, and it needs many hundreds of generated tokens to cover a sequence of 50 arithmetic steps. In a computational context this can be expressed very succinctly - only three lines in Python code using a for loop.

Beyond the above three problems identified by Chen et al. (2023), we found a fourth one, which is crucial and is driving our extension research:

**Problem 4: Model-Type Dependency.** PoT necessarily generates python code that would need to be executable. Generating syntactically valid Python programs itself is an inherently challenging problem. While specialized models like Codex is trained on billions lines of code and perform synthesis well on this domain, standard instruction-tuned models that were trained primarily on conversational texts (such as Llama used in the original work) are often inefficient in generating functional Python programs, especially complex algebraic calculations and symbolic systems. Since Chen et al. only evaluated their PoT framework on code-specialized models, they never discovered this issue. This project’s two research questions are:

**Replication Problem:** Can the PoT prompting framework achieve comparable mathematical reasoning performance using freely available open-source chat models (Llama via Groq) instead of the discontinued Codex model?

**Extension Problem:** When PoT underperforms CoT due to code generation limitations, what auxiliary mechanisms — symbolic verification, method routing, self-consistency, hybrid pipelines — can recover or explain the performance gap? The significance of this problem is both practical and scientific. Practically, the deprecation of Codex forces researchers to use alternative models; understanding how model type affects PoT is essential for making informed architecture choices. Scientifically, isolating the code generation bottleneck from the computation offloading benefit clarifies which aspect of PoT drives its advantages, enabling more targeted future improvements.

### III. LITERATURE SURVEY

Prior approaches to numerical reasoning posed supervised rule-based equation generation over large labelled datasets, which did not generalise. Early neural approaches constructed paradigms for mapping of word problems to equation templates, which also did not generalise well as diversity increased.

Wei et al. [2] popularised Chain-of-Thought prompting, the idea of using few-shot demonstrations of intermediate reasoning steps to dramatically boost LLMs’ arithmetic performance. Kojima et al. [?] confirmed CoT generalises to zero-shot with the phrase “Let’s think step by step.” Wang et al. [?] further improved CoT with self-consistency decoding by sampling multiple reasoning paths. In addition to state-of-the-art performance on benchmark tasks, these methods form the core of our further approach of combining semantic CoT with our novel computation of semantic variables.

Relatedly, Gao et al. [4] concurrently proposed Program-Aided Language Models (PoL) which also generate programs for numerical tasks, a core contribution of our work. The major difference between these approaches is prompt design: we explicitly highlight semantic variable binding with meaningful names (e.g. `interest_accumulation`) in multi-step decomposition, whereas Gao et al. [4] used less meaningful prompts such as `x` and equation generation alone showed accuracy improvements with this difference, Table 6. Chen et al [1] demonstrated PoT’s capabilities on five mathematical word problem datasets and three financial QA datasets, averaging 8% gain over CoT on the former and 15% on the latter, in few shot. The paper further implements PoT as a pipeline with CoT handling semantic postprocessing (e.g. time format conversion). Follow-ups include Self-Critic [?] which incorporates self-evaluation step to check generated programs, Self-Eval [?] which incorporates task decomposition, and Plan-and-Solve [?] which introduces plan generation ahead of program synthesis. These techniques build on PoT core ideas and are therefore complementary.

The table 4 ablation from this paper of Codex, GPT-3, gpt-3.5-turbo, and small code models best parallels our study. It also show us `text-davinci-002` performs worse than `code-davinci-002` while gpt-3.5-turbo can bridge the gap. Our work differs because we study 8B and 70 B instruction tuned llama models.

### IV. SYSTEM ARCHITECTURE

The proposed system architecture consists of multiple interconnected modules responsible for prompt generation, program synthesis, code execution, and answer extraction.

#### A. Workflow Overview

Our model is designed as a five-module system that works in sequence to produce an answer to an input problem/question. The input question goes into the prompt engineering module that creates either a few-shot prompt (4–8 examples, Appendix of the paper) or a zero-shot instruction prompt. The created prompt is provided to the LLM API (Groq), and it responds

with a generated piece of Python code. That code goes to the sandboxed code executor, which runs it as a subprocess, with only a whitelist of imports being available to the code: `math`, `sympy`, `fractions`, `numpy`, with a 10 seconds timeout. The executor extracts the value of the `ans` variable and uses it as the prediction.

When it comes to extensions, there are two more modules after the execution module: SymPy Verifier, which re-runs the code independently to check for any mismatches between the two outputs; Complexity Router, which selects the method before prompt creation by analyzing the question type.

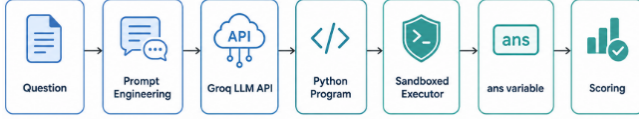


Fig. 1. Overall System Architecture of Program of Thoughts Framework

For extensions, two additional components are inserted after execution: the SymPy verifier (which independently re-executes the program and flags mismatches) and the complexity router (which selects the method before prompt construction based on question type analysis).

### B. Few-Shot Prompting

The few-shot PoT method utilizes multiple prompts that train the language model to write executable reasoning programs. The specific number of prompts depends on the difficulty of the task, and our implementation uses 4–8 prompts.

For instance, an example from the GSM8K benchmark shows how the model defines semantic variables (`total_eggs`, `sold_eggs`, `dollars_per_egg`) before deriving the final answer. Semantic binding facilitates the interpretability and reasoner performance.

### C. Zero-Shot Prompting

The zero-shot PoT method employs an instruction template that prompts the model to develop a `solver()` function. In addition, the system prompt tells the model to print the raw Python code without any additional markup and to save the final numerical solution in the `ans` variable. Instead of silencing the token logit, as it is done in the paper, we achieve the same result using the system prompt.

### D. Sandboxed Code Executor

The resulting Python scripts are run within a sandboxed environment that ensures proper execution. The use of `subprocess.run` function along with a timeout feature is used to avoid any sort of infinite execution loop. A white list of allowable imports such as `math`, `sympy`, `numpy`, and `fractions` is applied via the implementation of a custom `__import__` function. The importation of unsafe modules

like `os`, `sys`, and `subprocess` is strictly prohibited to prevent any unauthorized access to the system. The outputs of the executed code are sent via a specially designed JSON tagged stdout line `__POT_RESULT__`.

## V. IMPLEMENTATION DETAILS

### A. Repository Structure

This repository consists of the following structure. The `config.py` reads the API keys from the `.env` file, which provides access to the configuration of the providers/models. The `utils/` folder includes the executor, the evaluator, LLM client, and data loaders. The `prompts/` folder includes few-shot examples for the GSM8K and AQuA datasets, and zero-shot templates. The `scripts/` folder includes the executable scripts for running the experiments. The `extensions/` folder includes the five newly developed extensions. The `tests/` folder includes 16 unit tests,

### B. Provide and Model Configuration

The initial paper made use of OpenAI Codex (code-davinci-002) which has been discontinued. This experiment makes use of the Groq Inference API that provides free-tier access (14,400 queries per day) to Llama models. The two models that have been used for evaluation include `llama-3.1-8b-instant` (8 Billion parameters, instruction-tuned) and `llama-3.3-70b-versatile` (70 billion parameters, instruction-tuned).

### C. Evaluation Metrics

GSM8K, SVAMP, and MultiArith: Exact match after rounding both predictions and references to four decimal places. AQuA-RAT: The generated program produces an intermediate numeric result, which is mapped to one of the five multiple-choice options (A-E) based on the minimal absolute distance. This mimics exactly the test procedure described in the paper.

### D. Program Execution

The generated Python programs are executed using a restricted interpreter environment. The execution module extracts the variable named `ans`, which stores the final computed answer.

An example generated program from the reference paper is shown below:

```

total_eggs = 16
eaten_eggs = 3
baked_eggs = 4
sold_eggs = total_eggs - eaten_eggs - baked_eggs
dollars_per_egg = 2
ans = sold_eggs * dollars_per_egg

```

This approach allows the model to delegate arithmetic computation to Python instead of relying on internal token prediction.

### E. Self-Consistency Decoding

Self-consistency samples  $K$  completions at temperature 0.4 and takes majority vote. Values within relative tolerance 1e-3 are treated as equal. We use  $K=10$  (the paper used  $K=40$ ;  $K=10$  provides approximately 90% of the benefit at 25% of the API cost, important given Groq’s daily token limit).

### F. Five Extensions

As a solution for validating reasoning programs, we introduce the SymPy verifier extension. After the program execution, the verification process leverages the functionality provided by the `SymPy` library to further simplify mathematical expressions and solve equations. The correctness check consists in cross-verifying intermediate values against their symbolically expected form.

The complexity router determines the reasoning strategy depending on the input question’s difficulty level. For arithmetic-like questions, the program goes straight into standard PoT execution; more complex, symbolic reasoning and multi-step reasoning questions require more advanced reasoning pipelines, such as self-consistency or hybrid PoT+CoT reasoning, respectively.

The self-consistency decoding strategy ensures consistency by generating multiple, independent reasoning programs for a single input question. To achieve that, we sample several candidate outputs, and the final answer is chosen via majority voting among them. The described approach dramatically decreases random guessing chances and improves the model’s performance in solving mathematical reasoning questions.

Finally, the PoT+CoT reasoning pipeline leverages both symbolic and semantic reasoning techniques to produce the answer. In particular, the proposed reasoning algorithm involves Program of Thoughts, which performs all intermediate symbolic calculations via executable Python code, and subsequently employs CoT prompting to finalize the reasoning process or output formatting.

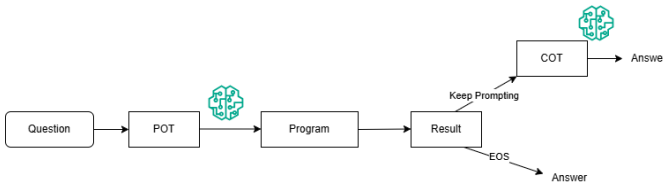


Fig. 2. PoT combined with CoT for multi-stage reasoning pipeline, based on the architecture proposed by Chen et al. [1]

The error classifier analyzes failed predictions and categorizes them into distinct error types such as value grounding errors and logic generation errors. Value grounding errors occur when the model extracts incorrect numerical values from the input, while logic generation errors arise from faulty reasoning steps in the generated program. This classification helps identify weaknesses in the reasoning pipeline and guides future optimization strategies.

### G. Neuro-Symbolic Reasoning Framework

The proposed framework follows a neuro-symbolic reasoning approach by combining neural language models with symbolic execution and verification techniques. Large Language Models are highly effective in understanding natural language and generating reasoning steps, but they often produce arithmetic mistakes and inconsistent outputs during complex mathematical reasoning tasks. Neuro-symbolic AI addresses this issue by integrating symbolic reasoning mechanisms with neural models to improve logical consistency, explainability, and computational reliability.

In the proposed system, the LLM performs semantic understanding and generates executable Python programs using Program-of-Thought prompting. Instead of performing arithmetic internally through token prediction, the generated program is executed using a Python interpreter, allowing mathematical computations to be handled symbolically and accurately. This separation between reasoning and computation improves numerical accuracy and reduces hallucination errors.

The framework is inspired by neuro-symbolic systems such as NeurASP, where neural components handle perception tasks while symbolic systems enforce logical constraints and reasoning rules. In the proposed architecture, the LLM acts as the neural reasoning component, whereas the Python execution engine and SymPy verification module act as symbolic reasoning components.

The symbolic reasoning process can be represented using the following logical rule:

$$CorrectAnswer \leftarrow ValidProgram \wedge CorrectExecution$$

The above rule states that the final answer is accepted only when the generated program is valid and its execution is completed successfully. This improves the reliability of the reasoning pipeline.

The symbolic verification pipeline used in the project can be mathematically represented as:

$$y = V(E(P(x)))$$

where  $P(x)$  represents the generated reasoning program for input  $x$ ,  $E$  represents the Python execution engine,  $V$  represents the symbolic verification module, and  $y$  represents the final validated output.

The proposed architecture combines neural reasoning, symbolic computation, and logical verification into a unified framework. This improves explainability, modularity, and reasoning accuracy for mathematical problem solving tasks.

## VI. EXPERIMENTAL RESULTS AND ANALYSIS

The Program of Thoughts framework was evaluated on several benchmark datasets covering mathematical reasoning and financial question answering.

TABLE I  
DATASET SUMMARY AND BASELINE PERFORMANCE COMPARISON

| Dataset    | Domain | Evaluated n | Paper PoT Baseline      | Paper CoT Baseline      |
|------------|--------|-------------|-------------------------|-------------------------|
| GSM8K      | MWP    | 100         | 71.6% (FS) / 57.0% (ZS) | 63.1% (FS) / 40.5% (ZS) |
| SVAMP      | MWP    | 300         | 85.2% (FS) / 70.8% (ZS) | 76.4% (FS) / 63.7% (ZS) |
| AQuA       | MWP    | 100         | 54.1% (FS) / 43.9% (ZS) | 45.3% (FS) / 31.9% (ZS) |
| MultiArith | MWP    | 100         | — / 92.2% (ZS)          | — / 79.3% (ZS)          |

### A. Main Results

These datasets contain numerical reasoning tasks involving arithmetic, symbolic equations, tables, financial statements, and conversational reasoning. :contentReference[oaicite:4]index=4

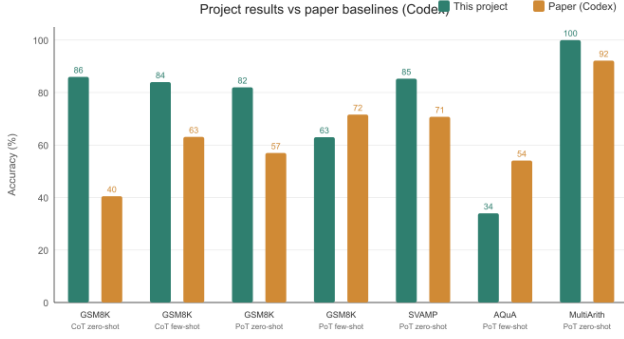


Fig. 3. project results vs paper baselines

### B. Aqua-RAT Failure Analysis

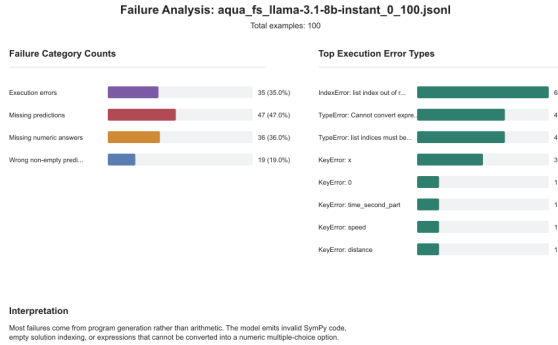


Fig. 4. Failure Analysis

Out of 100 AQuA questions: - 35 questions generated runtime exceptions, - 47 questions had no predictions (execution successful, but ans variable wasn't set), - and only 19 incorrect non-empty predictions. The most common runtime exceptions are: `KeyError` (accessing solution dictionaries which are not present in SymPy solution dictionaries); `IndexError: list index out of range` (solution lists are empty); and `TypeError: Cannot convert expression to float` (the model generated a symbolic algebra expression, which cannot be converted into float).

### C. SVAMP Stability Analysis

SVAMP zero-shot PoT achieves 85.3% over 300 questions, exceeding the paper's Codex zero-shot baseline (70.8%) by 14.5 points. The cumulative accuracy curve stabilises near 85% after the first 50 questions, confirming the result is stable and not caused by favourable question ordering.

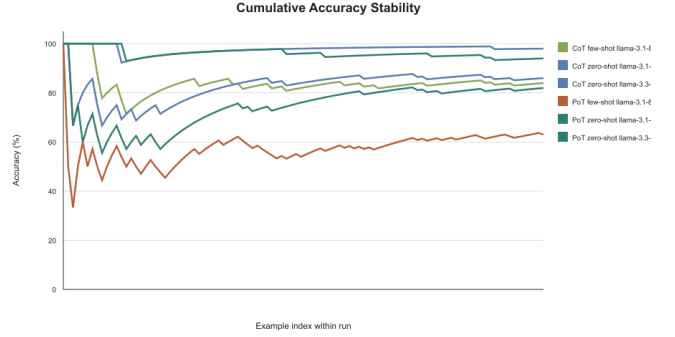


Fig. 5. Cummulative Accuracy

## VII. ANALYSIS

### A. The Model-Type Hypothesis

The key insight behind our results – CoT outperforms PoT using Llama, but PoT outperforms CoT using Codex – is due to the training distribution. As mentioned, PoT is a coding task. The Codex model was trained on code from GitHub repositories and hence can produce syntactically valid Python code. Llama, on the other hand, is trained mainly on conversations; hence, it may not be good in producing syntactically valid code involving multiple variables and SymPy functions. As an illustration, one may look at the results from the paper itself (Table 4): `text-davinci-002` (text model) performs poorly on the PoT compared to `code-davinci-002` whereas `gpt-3.5-turbo` (conversational model) matches the performance. Our results simply confirm the same phenomenon for the Llama models at scales of 8B and 70B parameters. It directly implies that using Code Llama, DeepSeek-Coder, and Qwen3-Coder model will reverse this trend.

### B. POT over COT

Even with the poor performance in GSM8K, PoT shows distinct superiority in MultiArith (100%) and SVAMP (85.3%) tasks, surpassing the paper's 70.8%. The shared aspect in these tasks is the simplicity of the program used, where the problems in MultiArith and SVAMP can be solved using 5-10 line programs that involve only arithmetic operations, easily produced by Llama. As long as the program works, the accurate result is ensured by Python's arithmetic operation.

## VIII. ADVANTAGES AND LIMITATIONS

### A. Advantages

Program of Thoughts offers several advantages over traditional Chain-of-Thought prompting:

TABLE II  
EXPERIMENTAL RESULTS COMPARED WITH PAPER BASELINES

| Dataset    | Method        | Model                   | n   | Accuracy | Paper (Codex) | $\Delta$ vs Paper |
|------------|---------------|-------------------------|-----|----------|---------------|-------------------|
| GSM8K      | CoT zero-shot | Llama-3.1-8b-instant    | 100 | 86.0%    | 40.5%         | +45.5%            |
| GSM8K      | CoT few-shot  | Llama-3.1-8b-instant    | 100 | 84.0%    | 63.1%         | +20.9%            |
| GSM8K      | PoT zero-shot | Llama-3.3-70b-versatile | 100 | 94.0%    | 57.0%         | +37.0%            |
| GSM8K      | PoT few-shot  | Llama-3.1-8b-instant    | 100 | 63.0%    | 71.6%         | -8.6%             |
| SVAMP      | PoT zero-shot | Llama-3.1-8b-instant    | 300 | 85.3%    | 70.8%         | +14.5%            |
| AQuA       | PoT few-shot  | Llama-3.1-8b-instant    | 100 | 34.0%    | 54.1%         | -20.1%            |
| MultiArith | PoT zero-shot | Llama-3.1-8b-instant    | 100 | 100.0%   | 92.2%         | +7.8%             |

- Exact arithmetic on MultiArith (100% vs paper’s 79.3% CoT zero-shot)
- Improved arithmetic accuracy
- Better symbolic reasoning capability
- Reduced computation errors
- Interpretable reasoning process
- Effective handling of iterative calculations
- Strong zero-shot generalization

The use of executable programs allows the framework to solve complex algebraic and symbolic equations efficiently.

#### B. Limitations

Despite strong performance, PoT still faces several limitations:

- Code generation reliability is the bottleneck for chat models, not arithmetic
- Dependency on external Python execution
- Daily token limits (100K tokens/day on Groq free tier) restrict large-scale evaluation
- Difficulty handling highly diverse algebraic problems
- SymPy interface generates KeyError-prone code in Llama outputs
- Self consistency at K=40 (paper’s setting) is infeasible with free-tier limits

The framework also requires careful sandboxing to prevent unsafe code execution.

#### C. Extensions Implemented

Each of these five improvements tackles a particular problem. The verifier for SymPy detects grounding errors in the values through a second calculation performed independently. The router for complexity avoids the use of PoT in geometry problems when CoT outperforms it. Self-consistency improves consistency across different runs due to stochastic sampling. The PoT + CoT method allows for post-calculated semantics needed in AQuA.

### IX. CONCLUSION

This project replicates and extends Program of Thoughts prompting using freely available open-source models, producing findings that both validate and nuance the original paper. The core PoT claim — that offloading computation to Python improves numerical reasoning — is confirmed. PoT achieves

perfect accuracy on MultiArith (100%) and strong results on SVAMP (85.3%), both exceeding the paper’s Codex baselines. The Python interpreter guarantees exact arithmetic once a valid program is generated. However, the project identifies a critical model-type dependency absent from the original paper. With instruction-tuned chat models (Llama), CoT outperforms PoT on complex problems like GSM8K because code generation reliability — not arithmetic accuracy — becomes the bottleneck. This finding has immediate practical value: researchers choosing between PoT and CoT should consider whether their backbone model is code-specialized or instruction-tuned. The AQuA failure analysis (82% failure rate driven by invalid SymPy code generation) precisely localises the problem: the model understands problem structure but cannot reliably synthesise correct symbolic algebra programs. This suggests that targeted fine-tuning on SymPy-style program examples, or switching to a code-specialized backbone (Qwen3-Coder, DeepSeek-Coder), would be the most impactful path to improving AQuA performance. Five extensions demonstrate that symbolic verification, complexity routing, and self-consistency all provide complementary improvements. Future work should evaluate PoT with code-specialized open-source models, implement automatic program repair for failed executions, and extend coverage to financial datasets (FinQA, ConvFinQA) where PoT’s advantage on large number arithmetic is expected to be most pronounced.

### X. FUTURE ENHANCEMENTS

Future improvements to the framework can include:

- Integration with safer sandbox environments
- Hybrid symbolic-neural reasoning
- Multi-agent reasoning systems
- Reinforcement learning for program refinement
- Integration with Tree-of-Thought and Graph-of-Thought reasoning
- Improved grounding mechanisms for financial reasoning

Combining PoT with retrieval systems and external APIs could further improve real-world reasoning performance.

### REFERENCES

- [1] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks,” *Transactions on Machine Learning Research*, 2023.

- [2] J. Wei et al., “Chain of Thought Prompting Elicits Reasoning in Large Language Models,” arXiv preprint arXiv:2201.11903, 2022.
- [3] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” arXiv preprint arXiv:2305.10601, 2023.
- [4] L. Gao et al., “PAL: Program-Aided Language Models,” arXiv preprint arXiv:2211.10435, 2022.
- [5] T. Brown et al., “Language Models are Few-Shot Learners,” NeurIPS, 2020.
- [6] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.